Machine Coding: Real-time Collaboration UI

Problem: Build a real-time collaboration UI: several people edit one document at the same time and see each other's changes, live cursors, presence (who is online) and typing indicators.
Tech stack: React (Vite, JavaScript), no extra libraries (default, not specified)
Time limit: 60 minutes (default, not specified)

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements**: functional, non-functional, clarifying questions, assumptions.
2. **List interactions / use cases**: every action and its expected result. This is your checklist.
3. **Design the architecture**: components / modules and who talks to whom.
4. **Design state**: minimum state, who owns it, what is derived.
5. **Design data structures and contracts**: shapes, ids, message/API payloads.
6. **Trace the data flow** for the main interaction end to end.
7. **Find the hard parts** and decide how to solve them before coding.
8. **Build in order**: simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases**: empty, loading, error, offline, conflicts.
10. **Wrap up**: trade-offs, what you would add with more time.

Time budget (60 minutes)

Phase	Time
Requirements, questions, assumptions	5 min
Use cases, architecture, state, message contract	8 min
Transport + single-tab editor	10 min
Sync edits across tabs + conflict rule	12 min
Presence, typing indicator, live cursors	12 min
Offline queue, caret fix, edge cases	8 min
Wrap-up and trade-offs	5 min

Tip: think out loud

Say every decision and its reason as you make it: "*I'll fake the server with BroadcastChannel so two tabs act as two users; the transport has a send/subscribe API so a WebSocket can replace it later.*" The interviewer grades the reasoning as much as the code. If you are stuck, say what you are trying and why.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A shared document made of paragraphs (blocks). Any participant can **edit, add and delete** blocks.
- Changes from one participant appear for everyone else **in real time**.
- **Presence**: avatars of everyone online in the room, including "you".
- **Live cursors**: see other people's mouse pointer with their name.
- **Activity indicator**: show which block someone is in and "X is typing...".
- **Connection status**: Live / Offline, with edits made offline synced on reconnect.
- A participant who joins late gets the **current document**.

Non-functional requirements

Area	What it means here
Performance	Cursor moves fire 100+ times/s: throttle to one message per frame. Unchanged merges must not re-render.
Scalability	Per-block messages (not the whole doc per keystroke); transport hidden behind an interface so a WebSocket server can replace it.
Error handling	Storage failures, offline network, duplicate or out-of-order messages, peers that crash without saying goodbye.
Accessibility / usability	Labelled textareas and buttons, `role="status"` connection badge, visible focus, cursors hidden from screen readers, reduced motion respected. Caret must not jump when someone else types.
Reusability	`useCollaboration` hook owns all real-time logic; components are dumb views. Pure merge logic in its own file.
Extensibility	New message types (comments, selections) plug into one `switch`. Conflict rule can be swapped for OT/CRDT later.

Clarifying questions to ask

1. Is there a real backend (WebSocket URL), or should I mock real-time? Can I use a library like Yjs?
2. What is the unit of editing: plain text paragraphs, rich text, or a canvas (Figma-like)?
3. How should **concurrent edits to the same paragraph** be resolved: last writer wins, or character-level merge?
4. Must the document survive a refresh / all users leaving?
5. Do we need authentication and real user names, or are guest identities fine?
6. How many concurrent users per document should we expect: 5, 50, 500?

Assumptions we build with

- **No backend**: real-time is mocked with `BroadcastChannel`; each browser tab is one user. The transport has a `send / subscribe / close` API so a WebSocket can replace it.
- Document = list of **plain text paragraphs**. Conflict rule = **last-writer-wins per paragraph** (version number + client id tie-break). Character-level merging (OT/CRDT) is out of scope and mentioned as a trade-off.
- Guest identity (random name + color) per page load; no auth.
- Document persisted to `localStorage` so it survives refresh.
- Small rooms (under ~20 users). Offline mode is simulated with a "Go offline" button.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Open the page (first user)	Doc loads from storage (or seed), "Only you here", badge "Live"
Open a second tab	Both tabs show 2 avatars; the new tab receives the latest document
Type in a paragraph	Text updates in the other tab on each keystroke; other tab shows "Guest Fox is typing…" on that block
Focus a paragraph without typing	Other tabs show a colored border + "Guest Fox is here"
Stop typing for ~2.5 s	Indicator changes back to "is here"
Add a paragraph	New empty block appears in all tabs; it is focused in the tab that created it
Delete a paragraph	Block disappears in all tabs
Move the mouse over the board	Other tabs see a named cursor at the same relative position
Mouse leaves the board	Cursor disappears for others
Two users edit the same block at the same moment	Both tabs end with the **same** text (deterministic winner)
Remote user types before my caret	My caret stays on the same character (does not jump to the end)
Click "Go offline", keep typing	Badge "Offline · N unsynced"; other tabs do not change
Click "Reconnect"	Queued edits are sent; missed edits are pulled; all tabs converge
Close or refresh a tab	Others remove that user immediately (leave message) or within 6 s (heartbeat timeout)
Delete every paragraph	Empty state: "This doc is empty. Add a paragraph to start."

---

3. Design the architecture

Module	Responsibility
`realtime/channel.js`	Transport: send and receive messages for a room (BroadcastChannel now, WebSocket later)
`utils/doc.js`	Pure logic: merge rule, snapshot merge, sorting visible blocks, presence timeout
`utils/identity.js`	Random guest identity (id, name, color)
`hooks/useCollaboration.js`	Owns doc, peers, cursors, connection state; handles messages, heartbeat, offline queue
`App.jsx`	Wires the hook to the views
`PresenceBar`	Avatars of everyone online
`ConnectionBadge`	Live/Offline status + toggle
`Board`	Document area, mouse tracking, add paragraph
`Block`	One paragraph textarea, caret preservation, activity tag
`CursorLayer`	Remote cursors drawn on top of the board

```
App
├── useCollaboration(roomId, me)  ──►  channel (BroadcastChannel / WebSocket)
│         └── utils/doc (pure merge + presence logic)
├── header
│   ├── PresenceBar
│   └── ConnectionBadge
└── Board
    ├── Block × N
    └── CursorLayer
```

Start simple: one hook, one transport, dumb components. The transport is split out from the start only because "mock now, WebSocket later" is a known requirement.

4. Design the state / data model

State or field	Purpose
`doc`	`{ [blockId]: block }` – the shared document, including deleted tombstones
`peers`	`{ [userId]: { user, blockId, typingAt, lastSeen } }` – who is online and where
`cursors`	`{ [userId]: { x, y } | null }` – last cursor position per peer
`online`	Is the (simulated) network up
`pending`	Number of blocks with unsynced offline edits (for the badge)

State or field	Purpose
`now`	Clock tick (every 2 s) to expire typing indicators and stale peers
`docRef`	Latest doc for message handlers without re-subscribing
`outboxRef`	`Map<blockId, op>` of offline edits (compacted)
`focusRef`	Which block I am in (sent in presence)

```
const [doc, setDoc] = useState(loadDoc)
const [peers, setPeers] = useState({})   // id -> { user, blockId, typingAt, lastSeen }
const [cursors, setCursors] = useState({}) // id -> { x, y } | null
const [online, setOnline] = useState(true)
const [pending, setPending] = useState(0)
const [now, setNow] = useState(Date.now)

const docRef = useRef(doc)            // latest doc for handlers
const outboxRef = useRef(new Map())   // blockId -> latest op while offline
```

Senior note:

- **Derived, not stored:** the ordered list of visible blocks (`visibleBlocks(doc)`), the list of peers, "who is in this block", "is typing" (`now - typingAt < 2500`), the badge text. Storing them would let them drift out of sync.
- **Ownership:** the hook owns all shared state; components only get props and callbacks. Each `Block` owns its own caret memory (`selectionRef`) because only it knows about the DOM caret. `Board` owns `newBlockId` (which block to autofocus) because it is pure UI.
- `doc` is a **map by id**, not an array: merges by id are O(1) and order comes from the `order` field, so two users adding blocks never fight over array positions.

---

5. Design the data structures and contracts

Block

```
{
  id: 'c1f0…',          // crypto.randomUUID(): stable, unique across tabs, no coordination
  text: 'Hello',
  order: 1730541600000, // creation time: sorts new blocks to the end
  version: 7,           // +1 on every local change to this block
  by: '9a2e…',          // client id of the last writer: tie-breaker
  deleted: false,       // tombstone instead of removing the key
}
```

Why stable random ids: two tabs can create blocks at the same time without asking a server for an id. Why a tombstone: if a delete were just "remove the key", a late edit for that block would bring it back and nobody could tell the block was deleted on purpose.

User

```
{ id: '9a2e…', name: 'Guest Fox', color: '#2563eb' }
```

Message contract (every message: `{ type, from, payload }`)

type	payload	Sent when	Receiver does
`hello`	`{ user, blockId, typing }`	On join and on reconnect	Adds peer, replies `snapshot` + `presence`
`snapshot`	`{ doc }`	In reply to `hello`	Merges every block (LWW)
`op`	`{ block }`	Each local edit/add/delete	Merges one block (LWW)
`presence`	`{ user, blockId, typing }`	Heartbeat (2 s), focus change, typing (throttled)	Updates `lastSeen`, `blockId`, `typingAt`
`cursor`	`{ id, pos: { x, y } | null }`	Mouse move (1 per frame), mouse leave	Updates cursor
`leave`	`{ id }`	Tab hidden/closed, unmount	Removes peer and cursor

Example `op`:

```
{ "type": "op", "from": "9a2e…",
  "payload": { "block": { "id": "welcome", "text": "Hello team", "order": 0,
                          "version": 8, "by": "9a2e…", "deleted": false } } }
```

The transport interface (what a WebSocket version must also provide):

```
const channel = createChannel(roomId, clientId)
channel.send(type, payload)
const unsubscribe = channel.subscribe((msg) => {})
channel.close()
```

---

6. Trace the data flow

User A types a letter in a paragraph; user B sees it.

```
A: keystroke in <textarea>
   │
   ▼
Block.handleChange → remember caret, lastTextRef = new value
   │
   ▼
useCollaboration.editBlock(id, text)
   ├─ build block { ...current, text, version: current.version + 1, by: A.id }
   ├─ commit(mergeBlock)  → docRef + setDoc  → A re-renders instantly (optimistic)
```

```
   ├─ send('op', { block })   (offline? → outbox Map instead)
   └─ throttled send('presence', { typing: true })
   │
   ▼  BroadcastChannel / WebSocket
B: channel.subscribe handler
   ├─ 'op'       → commit(d => mergeBlock(d, block))  (LWW: keep if newer)
   └─ 'presence' → peers[A].typingAt = B's Date.now()
   │
   ▼
B re-renders: Block gets new text
   ├─ useLayoutEffect: text changed remotely and I am focused → restore caret
   └─ tag "Guest Fox is typing…" with A's color border
   │
   ▼
2.5 s with no typing → next heartbeat tick → tag becomes "is here"
```

---

7. Important technical challenges

Challenge 1: Two people edit the same paragraph at the same time

Problem: A and B both have version 3 of a block. A types "x", B types "y" at the same moment. Each tab applies its own change first, then receives the other one. If the rule is "the last message I received wins", A ends with B's text and B ends with A's text: the tabs **disagree forever**.

Solution: Last-writer-wins with a **deterministic** order. Each change carries `version = previous + 1` and the writer's id. Higher version wins; equal versions are broken by comparing ids. Every tab applies the same rule, so every tab picks the same winner no matter the arrival order. The merge is idempotent (same op twice = no change) and commutative (any order = same result), which also makes snapshots and replays safe. Editing per **block** keeps conflicts small: two people in different paragraphs never conflict.

```
export const wins = (incoming, current) =>
  !current ||
  incoming.version > current.version ||
  (incoming.version === current.version && incoming.by > current.by)

// Same object back when nothing changed → React skips the render
export const mergeBlock = (doc, block) =>
  wins(block, doc[block.id]) ? { ...doc, [block.id]: block } : doc
```

Trade-off to say out loud: one of the two concurrent edits is lost. For character-level merging you need OT (Google Docs) or a CRDT (Yjs, Automerge).

Challenge 2: Presence: who is really online?

Problem: A user closes the laptop lid or the tab crashes. No "goodbye" message is sent, so their avatar and cursor stay forever.

Solution: Two layers. (1) Best effort: send `leave` on `pagehide` and on unmount. (2) Guarantee: everyone sends a **heartbeat** every 2 s; each receiver stamps `lastSeen` with **its own clock** and drops peers silent for 6

s (3 missed beats, so one slow message does not kick someone out). Using the receiver's clock avoids bugs from clock skew between machines.

```
const heartbeat = setInterval(() => {
  if (onlineRef.current) channel.send('presence', { user: me, blockId: focusRef.current, typing:
false })
  const t = Date.now()
  setNow(t)
  setPeers((prev) => pruneStale(prev, t, PRESENCE_TTL_MS))
}, HEARTBEAT_MS)

export function pruneStale(peers, now, ttl) {
  const alive = Object.entries(peers).filter(([, p]) => now - p.lastSeen < ttl)
  return alive.length === Object.keys(peers).length ? peers : Object.fromEntries(alive)
}
```

Challenge 3: Late joiners and going offline

Problem: B opens the doc after A has typed for 10 minutes; B must not start from an empty or stale document. And if A loses the network for a minute while typing, A's edits must not be lost, and A must catch up on what B wrote meanwhile.

Solution: On join (and on reconnect) send `hello`; peers reply with a full `snapshot`, merged block by block with the same LWW rule. While offline, document ops go to an **outbox keyed by block id**: 200 keystrokes in one paragraph become one pending op (only the latest version matters under LWW). Cursors and presence are dropped, since they are stale by the time we reconnect. On reconnect: flush the outbox, then `hello` to pull missed changes. Because the merge is idempotent, duplicates are harmless.

```
const send = useCallback((type, payload) => {
  if (onlineRef.current) channelRef.current?.send(type, payload)
  else if (type === 'op') {
    outboxRef.current.set(payload.block.id, payload) // compaction: latest op per block
    setPending(outboxRef.current.size)
  }
}, [])

// on reconnect
outboxRef.current.forEach((payload) => channelRef.current.send('op', payload))
outboxRef.current.clear()
channelRef.current.send('hello', { user: me, blockId: focusRef.current, typing: false })
```

Challenge 4: The caret jumps when someone else types, and cursors flood the channel

Problem (caret): I am typing in the middle of a paragraph. B adds a word at the start. React sets the textarea `value`, and the browser moves my caret to the **end**. My next keystroke lands in the wrong place.

Solution: Each `Block` remembers the last selection (on change and `onSelect`) and the text it last showed. In `useLayoutEffect` (before paint), if the text changed **remotely** and the textarea is focused, restore the caret: keep it if the text before it is unchanged, otherwise shift it by the length difference.

```
const shiftCaret = (pos, prev, next) =>
  prev.slice(0, pos) === next.slice(0, pos) ? pos : Math.max(0, pos + next.length - prev.length)

useLayoutEffect(() => {
  const prev = lastTextRef.current
  if (block.text === prev) return          // my own keystroke: browser caret is right
  lastTextRef.current = block.text
  const el = ref.current
  if (document.activeElement !== el || !selectionRef.current) return
  const [start, end] = selectionRef.current
  el.setSelectionRange(shiftCaret(start, prev, block.text), shiftCaret(end, prev, block.text))
}, [block.text])
```

Problem (cursors): `mousemove` fires 100+ times per second; sending each one floods the network and re-renders every peer.

Solution: Keep only the latest position and send it **once per animation frame**. Send `x` as a ratio of the board width so cursors line up when windows have different widths.

```
const moveCursor = useCallback((pos) => {
  const c = cursorRef.current
  c.pos = pos
  if (c.frame) return
  c.frame = requestAnimationFrame(() => {
    c.frame = 0
    send('cursor', { id: me.id, pos: c.pos })
  })
}, [send, me])
```

---

8. Implement in the correct order

#	Step	Working result
1	Static layout: header + list of textareas from a hard-coded doc	Page renders
2	`doc` state as a map + `visibleBlocks`; edit, add, delete locally	Single-user editor works
3	`createChannel` with BroadcastChannel; send `op` on edit, apply on receive	Two tabs sync (naive)
4	Add `version` / `by` and `mergeBlock` (LWW) + node assert check	Concurrent edits converge
5	`hello` → `snapshot` for late joiners; persist doc to localStorage	New tab gets current doc; refresh keeps it
6	Presence: identity, heartbeat, `pruneStale`, `leave` on pagehide; PresenceBar	Avatars appear/disappear
7	Focus + typing indicator on blocks (throttled presence)	"X is typing…" with colored border
8	Live cursors with rAF throttle + CursorLayer	Named cursors follow the mouse

#	Step	Working result
9	Caret preservation in `Block`	Caret stays put on remote edits
10	Offline toggle: outbox Map, badge, flush + `hello` on reconnect	Offline edits sync on reconnect
11	Empty state, accessibility labels, reduced motion, storage try/catch	Edge cases covered

9. Cover all states and edge cases

State / edge case	Handling
Initial (no saved doc)	Seed doc with a welcome paragraph
Initial (saved doc)	Load from localStorage; corrupt JSON or blocked storage → seed
Alone in the room	"Only you here", only your avatar
Syncing on join	`hello` → peers' snapshots merged; nothing blocks the UI (optimistic local doc)
Success / live	Badge "Live", edits appear on every keystroke
Empty document	"This doc is empty. Add a paragraph to start." + add button
Offline	Badge "Offline · N unsynced"; incoming messages ignored; edits queued per block
Reconnect	Flush outbox, pull snapshot; LWW makes duplicates harmless
Concurrent edit of same block	Deterministic winner (version, then client id)
Edit vs delete at the same time	LWW on the block: the later version wins (delete may be undone by a later edit)
Out-of-order / duplicate messages	Older or equal versions ignored; same reference returned → no re-render
Peer crashes / closes laptop	Removed after 6 s without heartbeat
Peer closes tab normally	`leave` on `pagehide` → removed at once
Remote edit while I type in same block	Caret restored with `shiftCaret`
Window sizes differ	Cursor `x` sent as ratio of width
Mouse leaves board	`pos: null` → cursor hidden
Many mouse events	One message per animation frame
Storage full	Write wrapped in try/catch; live sync still works
Clock skew between machines	Typing/lastSeen use the receiver's clock
React StrictMode double mount	Effect cleanup closes channel, clears interval, cancels rAF and resets the frame id

10. Final structure

```
_44_real_time_collaboration_ui/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx
    ├── App.css
    ├── realtime/
    │   └── channel.js          # transport (BroadcastChannel → WebSocket later)
    ├── hooks/
    │   └── useCollaboration.js # all shared state + sync logic
    ├── utils/
    │   ├── doc.js              # pure merge + presence logic
    │   ├── doc.check.js        # node assert self-check
    │   └── identity.js         # guest id, name, color
    └── components/
        ├── PresenceBar.jsx
        ├── ConnectionBadge.jsx
        ├── Board.jsx
        ├── Block.jsx
        └── CursorLayer.jsx
```

11. Full working code

Create the project with `npm create vite@latest _44_real_time_collaboration_ui -- --template react`, replace `src/` with the files below, run `npm run dev` and open the URL in **two tabs**. Run `node src/utils/doc.check.js` for the logic self-check.

src/realtime/channel.js

```
// Transport layer. BroadcastChannel syncs tabs of one browser (our mock "server").
// Swap the body for a WebSocket and keep the same { send, subscribe, close } API.
export function createChannel(roomId, clientId) {
  const bc = new BroadcastChannel(`collab:${roomId}`)
  const listeners = new Set()

  bc.onmessage = (event) => {
    const msg = event.data
    if (!msg || msg.from === clientId) return
    listeners.forEach((fn) => fn(msg))
  }

  return {
    send: (type, payload) => bc.postMessage({ type, from: clientId, payload }),
    subscribe: (fn) => {
      listeners.add(fn)
```

```
      return () => listeners.delete(fn)
    },
    close: () => bc.close(),
  }
}
```

src/utils/doc.js

```
// Pure document + presence logic. No React, so it is easy to test.

// Last-writer-wins per block: higher version wins, ties broken by client id
// so every tab picks the same winner no matter the arrival order.
export const wins = (incoming, current) =>
  !current ||
  incoming.version > current.version ||
  (incoming.version === current.version && incoming.by > current.by)

// Returns the same object when nothing changed, so React can skip the render.
export const mergeBlock = (doc, block) => (wins(block, doc[block.id]) ? { ...doc, [block.id]:
block } : doc)

export const mergeDoc = (doc, incoming) => Object.values(incoming).reduce(mergeBlock, doc)

export const visibleBlocks = (doc) =>
  Object.values(doc)
    .filter((b) => !b.deleted)
    .sort((a, b) => a.order - b.order || (a.id < b.id ? -1 : 1))

export function pruneStale(peers, now, ttl) {
  const alive = Object.entries(peers).filter(([, p]) => now - p.lastSeen < ttl)
  return alive.length === Object.keys(peers).length ? peers : Object.fromEntries(alive)
}

export const isTyping = (peer, now, windowMs) => now - peer.typingAt < windowMs
```

src/utils/doc.check.js

```
// Run with: node src/utils/doc.check.js
import assert from 'node:assert/strict'
import { mergeBlock, mergeDoc, pruneStale, visibleBlocks } from './doc.js'

const base = { id: 'b1', text: 'hi', order: 1, version: 1, by: 'a', deleted: false }
const fromA = { ...base, text: 'from A', version: 2, by: 'a' }
const fromB = { ...base, text: 'from B', version: 2, by: 'b' }

// Concurrent edits converge to the same winner in any order
const doc = { b1: base }
assert.equal(mergeBlock(mergeBlock(doc, fromA), fromB).b1.text, 'from B')
assert.equal(mergeBlock(mergeBlock(doc, fromB), fromA).b1.text, 'from B')

// Stale and duplicate ops are ignored and keep the same reference
const latest = mergeBlock(doc, fromA)
assert.equal(mergeBlock(latest, base), latest)
assert.equal(mergeBlock(latest, fromA), latest)
```

```
// Snapshot merge keeps the newer block from each side
const merged = mergeDoc({ b1: fromA }, { b1: base, b2: { ...base, id: 'b2', order: 2 } })
assert.equal(merged.b1.text, 'from A')
assert.deepEqual(visibleBlocks(merged).map((b) => b.id), ['b1', 'b2'])

// Tombstones are hidden
assert.equal(visibleBlocks(mergeBlock(merged, { ...fromA, version: 3, deleted: true })).length, 1)

// Presence TTL drops silent peers and keeps the reference when nobody left
const peers = { x: { lastSeen: 1000 }, y: { lastSeen: 9000 } }
assert.deepEqual(Object.keys(pruneStale(peers, 10000, 6000)), ['y'])
assert.equal(pruneStale(peers, 2000, 6000), peers)

console.log('doc.check.js: all assertions passed')
```

src/utils/identity.js

```
const ANIMALS = ['Fox', 'Owl', 'Panda', 'Tiger', 'Koala', 'Otter', 'Lynx', 'Heron']
const COLORS = ['#e11d48', '#2563eb', '#16a34a', '#d97706', '#7c3aed', '#0891b2', '#db2777',
'#4d7c0f']

const pick = (list) => list[Math.floor(Math.random() * list.length)]

// New identity per page load: a refreshed tab is a new participant and the old one times out.
export const createIdentity = () => ({
  id: crypto.randomUUID(),
  name: `Guest ${pick(ANIMALS)}`,
  color: pick(COLORS),
})
```

src/hooks/useCollaboration.js

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react'
import { createChannel } from '../realtime/channel'
import { mergeBlock, mergeDoc, pruneStale, visibleBlocks } from '../utils/doc'

const HEARTBEAT_MS = 2000
const PRESENCE_TTL_MS = 6000
const TYPING_THROTTLE_MS = 800
const STORAGE_KEY = 'collab:doc'

const SEED = {
  welcome: {
    id: 'welcome',
    text: 'Open this page in a second tab and start typing. Edits, cursors and presence sync
live.',
    order: 0,
    version: 1,
    by: 'seed',
    deleted: false,
  },
}
```

```
function loadDoc() {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY)) ?? SEED
  } catch {
    return SEED // corrupt or blocked storage: start from the seed
  }
}

export function useCollaboration(roomId, me) {
  const [doc, setDoc] = useState(loadDoc)
  const [peers, setPeers] = useState({}) // id -> { user, blockId, typingAt, lastSeen }
  const [cursors, setCursors] = useState({}) // id -> { x, y } | null
  const [online, setOnline] = useState(true)
  const [pending, setPending] = useState(0)
  const [now, setNow] = useState(Date.now)

  const channelRef = useRef(null)
  const docRef = useRef(doc) // latest doc for handlers, updated together with state
  const onlineRef = useRef(true)
  const outboxRef = useRef(new Map()) // blockId -> latest op while offline
  const focusRef = useRef(null)
  const lastTypingRef = useRef(0)
  const cursorRef = useRef({ frame: 0, pos: null })

  const commit = useCallback((update) => {
    const next = update(docRef.current)
    if (next === docRef.current) return
    docRef.current = next
    setDoc(next)
  }, [])

  const send = useCallback((type, payload) => {
    if (onlineRef.current) channelRef.current?.send(type, payload)
    else if (type === 'op') {
      // Only document ops survive offline; cursors and presence are stale by reconnect time
      outboxRef.current.set(payload.block.id, payload)
      setPending(outboxRef.current.size)
    }
  }, [])

  const sendPresence = useCallback(
    (typing = false) => send('presence', { user: me, blockId: focusRef.current, typing }),
    [send, me],
  )

  useEffect(() => {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(doc))
    } catch {
      /* storage full or blocked: live sync still works */
    }
  }, [doc])

  useEffect(() => {
    const channel = createChannel(roomId, me.id)
    const cursor = cursorRef.current
    channelRef.current = channel
```

```
    const touchPeer = ({ user, blockId, typing }) =>
      setPeers((prev) => {
        const t = Date.now()
        // Receiver's clock, not the sender's: avoids clock skew between machines
        const typingAt = typing ? t : (prev[user.id]?.typingAt ?? 0)
        return { ...prev, [user.id]: { user, blockId, typingAt, lastSeen: t } }
      })

    const removePeer = (id) => {
      setPeers((prev) => {
        const next = { ...prev }
        delete next[id]
        return next
      })
      setCursors((prev) => ({ ...prev, [id]: null }))
    }

    const unsubscribe = channel.subscribe(({ type, payload }) => {
      if (!onlineRef.current) return // simulated network drop
      switch (type) {
        case 'hello':
          touchPeer(payload)
          channel.send('snapshot', { doc: docRef.current })
          channel.send('presence', { user: me, blockId: focusRef.current, typing: false })
          break
        case 'snapshot':
          commit((d) => mergeDoc(d, payload.doc))
          break
        case 'op':
          commit((d) => mergeBlock(d, payload.block))
          break
        case 'presence':
          touchPeer(payload)
          break
        case 'cursor':
          setCursors((prev) => ({ ...prev, [payload.id]: payload.pos }))
          break
        case 'leave':
          removePeer(payload.id)
          break
      }
    })

    channel.send('hello', { user: me, blockId: null, typing: false })

    const heartbeat = setInterval(() => {
      if (onlineRef.current) channel.send('presence', { user: me, blockId: focusRef.current,
typing: false })
      const t = Date.now()
      setNow(t)
      setPeers((prev) => pruneStale(prev, t, PRESENCE_TTL_MS))
    }, HEARTBEAT_MS)

    const leave = () => channel.send('leave', { id: me.id })
    window.addEventListener('pagehide', leave)
```

```
    return () => {
      leave()
      clearInterval(heartbeat)
      window.removeEventListener('pagehide', leave)
      cancelAnimationFrame(cursor.frame)
      cursor.frame = 0
      unsubscribe()
      channel.close()
    }
  }, [roomId, me, commit])

  const updateBlock = useCallback(
    (id, changes) => {
      const current = docRef.current[id]
      const block = { ...current, ...changes, id, version: (current?.version ?? 0) + 1, by: me.id
}
      commit((d) => mergeBlock(d, block))
      send('op', { block })
    },
    [commit, send, me],
  )

  const editBlock = useCallback(
    (id, text) => {
      updateBlock(id, { text })
      const t = Date.now()
      if (t - lastTypingRef.current > TYPING_THROTTLE_MS) {
        lastTypingRef.current = t
        sendPresence(true)
      }
    },
    [updateBlock, sendPresence],
  )

  const addBlock = useCallback(() => {
    const id = crypto.randomUUID()
    updateBlock(id, { text: '', order: Date.now(), deleted: false })
    return id
  }, [updateBlock])

  const deleteBlock = useCallback((id) => updateBlock(id, { deleted: true }), [updateBlock])

  const focusBlock = useCallback(
    (id) => {
      focusRef.current = id
      sendPresence()
    },
    [sendPresence],
  )

  // Mouse moves fire ~100+/s; send at most one cursor update per animation frame
  const moveCursor = useCallback(
    (pos) => {
      const c = cursorRef.current
      c.pos = pos
      if (c.frame) return
      c.frame = requestAnimationFrame(() => {
```

```
        c.frame = 0
        send('cursor', { id: me.id, pos: c.pos })
      })
    },
    [send, me],
  )

  const setNetwork = useCallback(
    (next) => {
      onlineRef.current = next
      setOnline(next)
      if (!next) return
      outboxRef.current.forEach((payload) => channelRef.current.send('op', payload))
      outboxRef.current.clear()
      setPending(0)
      // Pull everything we missed; LWW merge makes replays and duplicates harmless
      channelRef.current.send('hello', { user: me, blockId: focusRef.current, typing: false })
    },
    [me],
  )

  const blocks = useMemo(() => visibleBlocks(doc), [doc])
  const peerList = useMemo(() => Object.values(peers), [peers])

  return {
    blocks,
    peers: peerList,
    cursors,
    online,
    pending,
    now,
    editBlock,
    addBlock,
    deleteBlock,
    focusBlock,
    moveCursor,
    setNetwork,
  }
}
```

src/components/PresenceBar.jsx

```
const initials = (name) => name.replace('Guest ', '').slice(0, 2).toUpperCase()

export default function PresenceBar({ me, peers }) {
  const people = [{ ...me, isMe: true }, ...peers.map((p) => p.user)]

  return (
    <div className="presence">
      <ul className="avatars" aria-label={`${people.length} people online`}>
        {people.map((u) => (
          <li key={u.id} className="avatar" style={{ '--color': u.color }} title={u.isMe ?
`${u.name} (you)` : u.name}>
            {initials(u.name)}
          </li>
```

```
        ))}
      </ul>
      <span className="presence-count">{peers.length === 0 ? 'Only you here' : `${people.length}
online`}</span>
    </div>
  )
}
```

src/components/ConnectionBadge.jsx

```
export default function ConnectionBadge({ online, pending, onToggle }) {
  const label = online ? 'Live' : pending > 0 ? `Offline · ${pending} unsynced` : 'Offline'

  return (
    <div className="connection">
      <span className={`badge ${online ? 'badge-online' : 'badge-offline'}`} role="status">
        {label}
      </span>
      <button type="button" className="btn" onClick={onToggle}>
        {online ? 'Go offline' : 'Reconnect'}
      </button>
    </div>
  )
}
```

src/components/Block.jsx

```
import { useLayoutEffect, useRef } from 'react'
import { isTyping } from '../utils/doc'

const TYPING_WINDOW_MS = 2500

// Keep a caret where it was when text before it is unchanged, else shift it by the length change
const shiftCaret = (pos, prev, next) =>
  prev.slice(0, pos) === next.slice(0, pos) ? pos : Math.max(0, pos + next.length - prev.length)

export default function Block({ block, editors, now, autoFocus, onEdit, onDelete, onFocus }) {
  const ref = useRef(null)
  const lastTextRef = useRef(block.text) // text this textarea last showed
  const selectionRef = useRef(null) // caret before React overwrote the value

  // A remote edit replaces the value and the browser jumps the caret to the end; put it back
  useLayoutEffect(() => {
    const prev = lastTextRef.current
    if (block.text === prev) return
    lastTextRef.current = block.text
    const el = ref.current
    if (document.activeElement !== el || !selectionRef.current) return
    const [start, end] = selectionRef.current
    el.setSelectionRange(shiftCaret(start, prev, block.text), shiftCaret(end, prev, block.text))
  }, [block.text])

  const rememberSelection = (e) => {
    selectionRef.current = [e.target.selectionStart, e.target.selectionEnd]
```

```
  }

  const handleChange = (e) => {
    lastTextRef.current = e.target.value
    rememberSelection(e)
    onEdit(block.id, e.target.value)
  }

  const typer = editors.find((p) => isTyping(p, now, TYPING_WINDOW_MS))
  const owner = typer ?? editors[0]

  return (
    <div className={`block ${owner ? 'block-shared' : ''}`} style={owner ? { '--color':
owner.user.color } : undefined}>
      <textarea
        ref={ref}
        className="block-input"
        value={block.text}
        rows={2}
        autoFocus={autoFocus}
        placeholder="Type something…"
        aria-label="Paragraph"
        onChange={handleChange}
        onSelect={rememberSelection}
        onFocus={() => onFocus(block.id)}
        onBlur={() => onFocus(null)}
      />
      <button type="button" className="block-delete" aria-label="Delete paragraph" onClick={() =>
onDelete(block.id)}>
        ×
      </button>
      {owner && (
        <span className="block-tag">
          {owner.user.name} {typer ? 'is typing…' : 'is here'}
        </span>
      )}
    </div>
  )
}
```

src/components/CursorLayer.jsx

```
export default function CursorLayer({ peers, cursors }) {
  return (
    <div className="cursor-layer" aria-hidden="true">
      {peers.map(({ user }) => {
        const pos = cursors[user.id]
        if (!pos) return null
        return (
          <div key={user.id} className="cursor" style={{ left: `${pos.x * 100}%`, top: pos.y, '--
color': user.color }}>
            <svg width="16" height="16" viewBox="0 0 16 16">
              <path d="M1 1l5 14 2-6 6-2z" fill="var(--color)" stroke="#fff" strokeWidth="1.2" />
            </svg>
            <span className="cursor-name">{user.name}</span>
```

```
          </div>
        )
      })}
    </div>
  )
}
```

src/components/Board.jsx

```
import { useMemo, useRef, useState } from 'react'
import Block from './Block'
import CursorLayer from './CursorLayer'

export default function Board({ blocks, peers, cursors, now, onEdit, onAdd, onDelete, onFocus,
onCursor }) {
  const boardRef = useRef(null)
  const [newBlockId, setNewBlockId] = useState(null)

  const editorsByBlock = useMemo(() => {
    const map = {}
    peers.forEach((p) => {
      if (p.blockId) (map[p.blockId] ??= []).push(p)
    })
    return map
  }, [peers])

  // x as a ratio so cursors line up across different window widths; y in px from the board top
  const handleMove = (e) => {
    const rect = boardRef.current.getBoundingClientRect()
    onCursor({ x: (e.clientX - rect.left) / rect.width, y: e.clientY - rect.top })
  }

  return (
    <main className="board" ref={boardRef} onMouseMove={handleMove} onMouseLeave={() =>
onCursor(null)}>
      {blocks.length === 0 && <p className="empty">This doc is empty. Add a paragraph to start.
</p>}
      {blocks.map((block) => (
        <Block
          key={block.id}
          block={block}
          editors={editorsByBlock[block.id] ?? []}
          now={now}
          autoFocus={block.id === newBlockId}
          onEdit={onEdit}
          onDelete={onDelete}
          onFocus={onFocus}
        />
      ))}
      <button type="button" className="btn btn-add" onClick={() => setNewBlockId(onAdd())}>
        + Add paragraph
      </button>
      <CursorLayer peers={peers} cursors={cursors} />
    </main>
```

```
  )
}
```

src/App.jsx

```
import './App.css'
import Board from './components/Board'
import ConnectionBadge from './components/ConnectionBadge'
import PresenceBar from './components/PresenceBar'
import { useCollaboration } from './hooks/useCollaboration'
import { createIdentity } from './utils/identity'

const ROOM_ID = 'design-doc'
const me = createIdentity() // one participant per page load

export default function App() {
  const collab = useCollaboration(ROOM_ID, me)

  return (
    <div className="app">
      <header className="topbar">
        <div>
          <h1>Design doc</h1>
          <p className="subtitle">
            You are <strong style={{ color: me.color }}>{me.name}</strong>
          </p>
        </div>
        <PresenceBar me={me} peers={collab.peers} />
        <ConnectionBadge
          online={collab.online}
          pending={collab.pending}
          onToggle={() => collab.setNetwork(!collab.online)}
        />
      </header>
      <Board
        blocks={collab.blocks}
        peers={collab.peers}
        cursors={collab.cursors}
        now={collab.now}
        onEdit={collab.editBlock}
        onAdd={collab.addBlock}
        onDelete={collab.deleteBlock}
        onFocus={collab.focusBlock}
        onCursor={collab.moveCursor}
      />
    </div>
  )
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
```

```
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/index.css

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: #f8fafc;
  color: #0f172a;
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
}
```

src/App.css

```
.app {
  max-width: 820px;
  margin: 0 auto;
  padding: 24px 16px 64px;
}

.topbar {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 20px;
}

.topbar h1 {
  margin: 0;
  font-size: 1.5rem;
}

.subtitle {
  margin: 4px 0 0;
  color: #64748b;
  font-size: 0.9rem;
}

.presence {
  display: flex;
  align-items: center;
```

```
  gap: 10px;
}

.avatars {
  display: flex;
  margin: 0;
  padding: 0;
  list-style: none;
}

.avatar {
  display: grid;
  place-items: center;
  width: 32px;
  height: 32px;
  margin-left: -8px;
  border: 2px solid #fff;
  border-radius: 50%;
  background: var(--color);
  color: #fff;
  font-size: 0.7rem;
  font-weight: 700;
}

.avatar:first-child {
  margin-left: 0;
}

.presence-count {
  color: #475569;
  font-size: 0.85rem;
}

.connection {
  display: flex;
  align-items: center;
  gap: 8px;
}

.badge {
  padding: 4px 10px;
  border-radius: 999px;
  font-size: 0.8rem;
  font-weight: 600;
}

.badge-online {
  background: #dcfce7;
  color: #166534;
}

.badge-offline {
  background: #fee2e2;
  color: #991b1b;
}

.btn {
```

```
  padding: 6px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #fff;
  font: inherit;
  font-size: 0.85rem;
  cursor: pointer;
}

.btn:hover {
  background: #f1f5f9;
}

.btn:focus-visible,
.block-delete:focus-visible {
  outline: 2px solid #2563eb;
  outline-offset: 2px;
}

.board {
  position: relative;
  display: flex;
  flex-direction: column;
  gap: 12px;
  min-height: 60vh;
  padding: 24px;
  border: 1px solid #e2e8f0;
  border-radius: 12px;
  background: #fff;
  box-shadow: 0 1px 3px rgb(0 0 0 / 6%);
}

.empty {
  margin: 0;
  color: #64748b;
}

.block {
  position: relative;
  border: 2px solid transparent;
  border-radius: 8px;
  transition: border-color 150ms;
}

.block-shared {
  border-color: var(--color);
}

.block-input {
  display: block;
  width: 100%;
  min-height: 3em;
  padding: 10px 36px 10px 12px;
  border: 1px solid #e2e8f0;
  border-radius: 6px;
  font: inherit;
  line-height: 1.5;
```

```
  resize: vertical;
  field-sizing: content;
}

.block-input:focus {
  outline: none;
  border-color: #94a3b8;
}

.block-delete {
  position: absolute;
  top: 6px;
  right: 6px;
  width: 24px;
  height: 24px;
  border: none;
  border-radius: 4px;
  background: transparent;
  color: #94a3b8;
  font-size: 1.1rem;
  cursor: pointer;
}

.block-delete:hover {
  background: #fee2e2;
  color: #b91c1c;
}

.block-tag {
  position: absolute;
  top: -11px;
  left: 10px;
  padding: 1px 8px;
  border-radius: 4px;
  background: var(--color);
  color: #fff;
  font-size: 0.7rem;
  font-weight: 600;
}

.btn-add {
  align-self: flex-start;
}

.cursor-layer {
  position: absolute;
  inset: 0;
  overflow: hidden;
  pointer-events: none;
}

.cursor {
  position: absolute;
  transition: left 80ms linear, top 80ms linear;
}

.cursor-name {
```

```
  display: inline-block;
  margin-left: 8px;
  padding: 1px 6px;
  border-radius: 4px;
  background: var(--color);
  color: #fff;
  font-size: 0.7rem;
  white-space: nowrap;
}

@media (prefers-reduced-motion: reduce) {
  .cursor,
  .block {
    transition: none;
  }
}
```

---

12. Wrap-up

Edge cases handled

- Concurrent edits converge to the same text in every tab (deterministic LWW per block).
- Late joiners get a snapshot; refresh keeps the doc (localStorage).
- Offline edits are queued, compacted per block and synced on reconnect; missed edits are pulled.
- Crashed peers time out; normal leaves are instant.
- Duplicate and out-of-order messages are ignored without re-rendering.
- Caret stays in place on remote edits; cursors are throttled and width-independent.
- Empty doc, storage errors, StrictMode double mount, reduced motion.

Trade-offs to mention

- **LWW per paragraph vs OT/CRDT:** LWW is simple and correct (everyone converges) but loses one of two concurrent edits to the same paragraph. Real editors use OT (server transforms ops) or CRDTs like Yjs (character-level, offline-first). I would use Yjs in production rather than write my own.
- **BroadcastChannel vs WebSocket:** BroadcastChannel only connects tabs of one browser; it is a mock. The transport interface is the seam for a WebSocket client.
- **Peer-to-peer snapshot:** every peer replies to `hello` with the full doc, so N peers send N snapshots. A server would hold the authoritative doc and send exactly one.
- **Whole block per keystroke:** simple and fine for paragraphs; for long blocks you would send diffs or CRDT updates.
- **Edit-vs-delete:** a later edit resurrects a deleted block. Acceptable here; a product decision.
- **Optimistic updates:** local edits apply instantly; there is no server rejection path because there is no server.

What to add with more time

- **Tests:** unit tests for `doc.js` in Vitest (already have node asserts), React Testing Library for Block caret logic, Playwright with two browser contexts for real sync.
- **Real backend:** WebSocket server with rooms, auth, authoritative doc, version history, reconnect with exponential backoff.
- **CRDT:** Yjs + y-websocket for character-level merging and undo/redo per user.
- **Features:** remote text selections (highlight what others selected), comments, block reordering with fractional indexing, names from auth, "follow user" mode.
- **Performance:** memoize `Block` with `React.memo`, virtualize very long docs, batch ops every 50 ms on slow networks.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you mock the server with BroadcastChannel instead of building a WebSocket server?

In 60 minutes the UI is what is being judged. BroadcastChannel gives real message passing between tabs with zero setup, so two tabs behave like two users. The transport has `send/subscribe/close`, so a WebSocket version is a drop-in replacement and nothing else changes.

Q2. Why did you split the document into blocks instead of one big text?

Conflicts are resolved per block, so two people editing different paragraphs never conflict. It also keeps messages small: one paragraph per op, not the whole document. Google Docs-like character merging needs OT/CRDT, which is out of scope.

State design

Q3. Why is the doc a map by id and not an array?

Merging is O(1) by id, and order comes from a field, not from array index. With an array, two users inserting at index 2 at the same time would shift each other's items and diverge. Ids also give React stable keys.

Q4. Why do you keep `docRef` next to `doc` state?

Message handlers are subscribed once in an effect; they would see a stale `doc` from the first render. `commit()` updates the ref and the state together, so handlers (like replying to `hello` with a snapshot) always read the latest doc without re-subscribing the channel.

Q5. Why don't you store `isTyping` or the sorted block list in state?

They are derived: `visibleBlocks(doc)` and `now - typingAt < 2500`. Stored copies can drift from the source. Derivation with `useMemo` is cheap.

Challenge 1: conflicts

Q6. Why not just use timestamps (`Date.now()`) to decide the winner?

Clocks on different machines differ by seconds. A user with a fast clock would always win, even with older edits. A per-block version counter only depends on what the writer has seen, and the id tie-break makes the result deterministic.

Q7. Two users edit the same paragraph at the same time. Whose text wins and is that acceptable?

Both create version n+1; the one with the larger client id wins in every tab, so everyone converges. One edit is lost. For paragraphs typed by different people that is usually acceptable in an interview scope; for a product I would switch to a CRDT (Yjs) that merges at character level.

Q8. How is OT different from a CRDT?

OT sends operations ("insert 'a' at 5") and a server transforms concurrent ops against each other so they still apply correctly. CRDTs give every character a unique id and position so any order of merges gives the same result without a central server. OT needs a server; CRDTs work peer-to-peer and offline.

Challenge 2: presence

Q9. Why a heartbeat if you already send `leave` on `pagehide`?

`pagehide` does not fire on crashes, killed processes, lost Wi-Fi or a closed laptop. The heartbeat plus a 6 s timeout is the guarantee; `leave` just makes the normal case instant.

Q10. Why is the timeout 3× the heartbeat interval?

One delayed or dropped heartbeat should not make a user flicker out of the list. Three missed beats is a common rule of thumb balancing quick cleanup and stability.

Challenge 3: offline / late joiners

Q11. What if a user is offline for an hour and makes 5,000 keystrokes?

The outbox is a `Map` keyed by block id, so it holds at most one op per edited block: the latest. On reconnect we send those few ops, then pull a snapshot. LWW guarantees the end state is the same as replaying all 5,000.

Q12. What if the snapshot is huge (a 10,000-block doc)?

Peer-to-peer every peer sends the full doc, which does not scale. With a server: send one snapshot from the server, paginate or stream it, or send only blocks changed since the client's last known version (a version vector or server sequence number).

Challenge 4: caret and cursors

Q13. Why `useLayoutEffect` and not `useEffect` for the caret?

`useLayoutEffect` runs after the DOM update but before paint, so the user never sees the caret jump to the end and back. With `useEffect` there can be a visible flicker.

Q14. Why throttle cursors with `requestAnimationFrame` instead of lodash throttle?

The screen cannot show more than one position per frame, so sending more is waste. rAF aligns with the display rate, needs no dependency, and pauses in background tabs automatically. A WebSocket version might go even lower (e.g. 20/s) to save bandwidth.

Q15. Why send the cursor x as a ratio and y in pixels?

Users have different window widths; text wraps differently and a pixel x would point at different content. A ratio keeps the cursor at the same relative spot. Vertical layout is mostly the same block stack, so pixels from the board top are close enough. A precise version would anchor cursors to a block id plus an offset.

Implementation steps

Q16. What happens in React StrictMode where effects run twice?

Mount → cleanup → mount. Cleanup sends `leave`, closes the channel, clears the interval and cancels the pending rAF (and resets the frame id, otherwise cursor updates would never be scheduled again). The second mount sends a fresh `hello`, so behavior is correct.

Q17. Why does each page load get a new identity instead of saving it in sessionStorage?

"Duplicate tab" copies sessionStorage, so two tabs would share one id and ignore each other's messages. A new id per load is simpler and correct; the old identity from a refresh times out after 6 s. With auth the user id would come from the server.

Performance

Q18. What if there are 500 users in the document?

Presence and cursors become the bottleneck (500 × 60 cursor msgs/s). Fixes: server fan-out with batching (one combined cursor message every 50 ms), only show cursors of users in your viewport, cap the avatar list ("+480"), send presence less often, and `React.memo` on `Block` and cursors so only changed ones re-render.

Q19. Every keystroke re-renders the whole Board. Is that a problem?

For tens of paragraphs no. For large docs: wrap `Block` in `React.memo` (props are stable because callbacks are `useCallback` and unchanged blocks keep their object reference thanks to `mergeBlock`), and virtualize the list.

Trade-offs / "why not X?"

Q20. Why not use Yjs or Liveblocks right away?

In production I would: they solve conflicts, presence and offline better than hand-written code. In an interview the goal is to show I understand the problems they solve (ordering, idempotency, presence liveness, caret mapping), so I build a small version and name the library as the next step.

Q21. Why not put the doc in Redux or Context?

Only one tree uses it and the hook already isolates it. Context would re-render every consumer on each keystroke anyway. If many distant components needed it, a store with selectors (Zustand/Redux) would be the next step.

Q22. What if the WebSocket is slow (500 ms latency)?

Local edits are optimistic, so typing still feels instant. Remote edits arrive late, which raises the chance of same-block conflicts; that is where a CRDT pays off. Show a "Syncing…" state if acks are pending too long, and batch ops instead of sending one per keystroke.